

信息学竞赛 AI 自动造数据与数据校验 工具

完整项目设计报告

专业实习小学期项目设计阶段

版本：V1.0

项目类型：中学 / 信息学竞赛工具

设计目标：指导 MVP 开发、答辩演示与需求二次确认

编制日期：2026 年 7 月 9 日

目录

1	项目背景与建设目标	3
1.1	建设目标	3
2	用户角色与使用场景	3
2.1	用户角色	4
2.2	典型使用场景	4
3	需求分析	4
3.1	功能需求	5
3.2	非功能需求	5
3.3	需求边界说明	6
4	Polygon-like 总体业务流程设计	6
4.1	流程说明	7
5	系统架构设计	7
5.1	分层架构	7
5.2	Mermaid 架构图描述	8
6	智能体工作流设计	8
6.1	工作流原则	8
6.2	节点设计	9
7	Docker 沙箱设计	9
7.1	镜像内容	9
7.2	编译流程	10
7.3	运行流程	10
7.4	安全限制	10
7.5	错误分类	10

8 内存控制设计	11
8.1 资源限制字段	11
8.2 失败停止策略	11
9 jngen 数据结构生成设计	12
10 testlib validator 设计	13
10.1 validator 职责	13
10.2 与基础校验器的关系	13
11 数据模型设计	13
12 API 设计	14
13 界面设计	15
13.1 前端输入控件设计	16
14 应用功能边界	17
14.1 建议纳入应用的功能	17
14.2 暂不纳入应用需要二次确认的功能	17
15 验收标准	18
16 推荐技术栈	19
17 项目目录结构建议	19
18 小组分工	20
19 开发里程碑	20
20 参考资料与当前目录依据	21

1 项目背景与建设目标

专业实习课程要求项目组完成组队选题、项目设计、项目开发、项目提交与项目答辩。课程考查文件明确指出，项目设计部分占总成绩 20%，设计报告需要体现项目业务逻辑结构清晰、功能完善、界面设计合理美观，并能够反馈需求方进行二次需求确认。当前报告面向项目设计阶段，目标是把需求、业务流程、系统架构、模块职责、界面方案、MVP 边界和验收标准沉淀为可执行的开发依据。

本项目的题目来源为“信息学竞赛 AI 自动造数据与数据校验工具”。原始需求聚焦中小学信息学竞赛训练场景，痛点包括人工造数据效率低、边界数据覆盖不全、测试数据容易出错、标准输出生成繁琐以及数据校验不足。项目目标是借助 AI 分析与规划能力，在本地实现一个 Polygon-like 的单题测试数据生成与验题流程。

本项目不是直接对接在线评测系统，也不是完整复制 Codeforces Polygon。MVP 阶段只围绕单道题完成数据生成、验证、标准输出生成、质量报告与本地导出。系统默认输入不是完整题面，而是用户导入的 C++ 标准程序、数据范围、数量规模和难度等级。AI 负责分析、规划、生成代码草稿和修复建议；Docker、testlib validator、jngen generator 和用户标程负责确定性执行与校验。

1.1 建设目标

1. 建立清晰的单题数据生成业务流程，覆盖输入配置、标程导入、数据结构识别、测试点计划、生成器与校验器生成、Docker 执行、报告与导出。
2. 建立安全可控的执行机制，所有不可信 C++ 代码必须在 Docker 沙箱内编译和运行，禁止在宿主机直接执行。
3. 建立以 testlib validator 为核心的数据合法性校验机制，确保最终数据包中的每个 .in 文件都通过严格验证。
4. 建立以用户导入标程为唯一答案来源的标准输出生成机制，不在 MVP 中由 AI 自动生成标准程序。
5. 建立面向需求方的二次确认闭环，把不确定输入结构、数据范围表达方式、OJ 兼容、课堂练习、模拟比赛等问题单独列出。
6. 建立可指导后续 MVP 开发的技术方案，包括 FastAPI、React + Vite、SQLite、Docker、Dify Python client wrapper、testlib.h、jngen.h、本地文件存储和 zip 导出。

2 用户角色与使用场景

2.1 用户角色

角色	主要目标	关注点
信息学教师 / 出题人	为单道训练题快速生成可靠测试数据	数据边界覆盖、格式正确、输出可信、导出方便
竞赛训练负责人	批量准备训练材料并检查数据质量	流程可控、报告可读、失败原因明确
项目组开发人员	根据设计报告开发 MVP	模块边界、接口定义、数据模型、执行安全
需求方 / 学校联系人	判断方案是否符合教学训练场景	业务逻辑、功能完整度、界面是否易用、后续需求边界

2.2 典型使用场景

1. 教师创建单题项目，填写项目名称和说明。
2. 教师导入 C++ 标准程序，系统在 Docker 中编译，确认标程可执行。
3. 教师填写数据范围文本、数量规模数值，并从单选下拉框中选择难度等级。数据范围文本可采用类似 Constraints 的描述，变量名需与标程代码保持一致。
4. 系统调用 Dify / Mock Dify 工作流，分析标程输入读取逻辑并生成输入结构草案、数据结构识别结果和测试点计划。
5. 系统生成 generator.cpp 和 validator.cpp 草稿，用户可预览并人工确认。
6. 系统在 Docker 中编译 generator、validator 和 solution，批量生成 .in 文件。
7. 系统使用 testlib validator 严格校验每个输入，过滤不合法、重复或超限测试点。
8. 系统使用用户标程生成 .out 文件，并记录运行时间、内存、退出码和文件大小。
9. 系统生成质量报告，展示数据结构覆盖、边界覆盖、运行日志和需要人工确认的问题。
10. 用户导出本地 zip 数据包，用于后续教学、训练或进一步人工整理。

3 需求分析

3.1 功能需求

功能模块	功能说明
单题项目管理	创建、查看、编辑单题项目，保存项目状态和基础配置。
标程导入与编译	上传或粘贴 C++ 标准程序，使用 Docker 编译，记录编译结果和错误日志。
输入配置管理	通过多行文本框录入类似 Constraints 的数据范围说明，变量名需与标程代码保持一致；通过单行文本框录入测试点数量等数值规模；通过单选下拉框选择难度等级；不要求用户输入完整题面。
Dify workflow	分析标程读取逻辑，结合输入配置输出输入结构草案、数据结构识别和测试点计划。
测试点计划	每个测试点包含 category、scale、purpose、seed、resource limit。
generator 生成	生成基于 jngen 的 generator.cpp 草稿，支持 seed 和 case_id。
validator 生成	生成基于 testlib 的 validator.cpp 草稿，严格检查输入格式、范围和结构约束。
Docker 编译运行	编译和运行 generator、validator、solution，限制 CPU、内存、时间和网络。
输入校验	对每个 .in 执行文件大小检查、基础格式检查、testlib validator 校验、重复检测和资源限制检查。
答案生成	使用用户标程生成 .out，记录 exit code、runtime、memory、stdout size、stderr、超时和内存超限状态。
质量报告	生成 report.md / 页面报告，展示配置摘要、测试点计划、验证结果、执行资源和失败原因。
zip 导出	导出本地通用数据包，包括输入输出文件、generator.cpp、validator.cpp、solution.cpp、metadata.json 和 report.md。

3.2 非功能需求

- 安全性：不可信代码必须在 Docker 中编译和运行，禁止网络访问，只挂载临时工作目录。

- 可复现性：generator 必须支持 seed 和 case_id，测试点生成过程应可追踪。
- 可观测性：Dify 调用、Docker 编译、运行日志、validator 失败原因和标程失败原因都必须记录。
- 可维护性：后端使用 FastAPI 分层设计，前端使用 React + Vite，数据持久化使用 SQLite。
- 可解释性：AI 推断出的输入结构和隐含约束必须展示不确定字段，不能直接替代人工确认。
- 性能边界：系统必须配置时间、内存、文件大小、总数据包大小和最大重试次数。

3.3 需求边界说明

当前应用优先覆盖单题数据生成、输入校验、标准输出生成、质量报告和 zip 导出闭环。暂不纳入应用且需要二次确认的事项集中列于“应用功能边界”章节，避免在需求分析阶段过早承诺 OJ 兼容、课堂练习、模拟比赛、SPJ、交互题等未明确需求。

4 Polygon-like 总体业务流程设计

本项目参考 Polygon 的出题验题流程，但只保留 MVP 所需的本地单题流程。Polygon 中 validator、generator、checker、solutions 和 tests 形成闭环，输出可由标准解自动生成。本项目继承这一思想：generator 只负责生成输入，validator 负责严格校验输入，solution 负责生成输出，系统记录全过程并导出数据包。

创建单题项目

- > 标程导入
- > 填写数据范围文本
- > 填写数量规模数值
- > 选择难度等级下拉选项
- > Dify 分析标程输入结构与约束草案
- > 数据结构识别
- > 测试点计划
- > 生成 generator.cpp
- > 生成 validator.cpp
- > Docker 编译
- > generator 生成 input
- > validator 校验 input
- > main solution 生成 output
- > 记录资源使用
- > 生成 report

-> 导出 zip

4.1 流程说明

- 1. 输入配置阶段以标程代码、数据范围文本、数量规模数值和难度等级下拉选项为入口，不要求完整题面。
- 2. 数据范围使用多行文本框填写，形式可接近竞赛题面中的 Constraints，页面必须提示变量名与标程代码保持一致。
- 3. 数量规模使用单行文本框填写测试点数量等数值，难度等级使用单选下拉框选择。
- 4. AI 分析阶段只产生草案和建议，不能替代确定性校验。
- 5. 代码生成阶段输出 generator.cpp 和 validator.cpp，用户可人工检查。
- 6. 执行阶段统一进入 Docker 沙箱，禁止宿主机直接运行用户代码。
- 7. 验证阶段以 testlib validator 为主，系统基础检查为辅。
- 8. 输出阶段只使用用户导入的标程生成 .out。
- 9. 报告阶段展示成功结果、失败原因和二次确认问题。

5 系统架构设计

5.1 分层架构

层次	职责
前端 UI 层	项目列表、输入配置、标程导入、资源限制、智能体流程、测试点计划、代码预览、数据预览、报告与导出。
后端 API 层	使用 FastAPI 提供项目管理、输入配置管理、标程管理、资源限制、Dify 工作流、代码生成、Docker 执行、数据导出接口。
Dify 智能体工作流层	通过 Python 封装 DifyWorkflowClient；优先参考本地 dify 文件夹；不存在可用服务时使用 MockDifyWorkflowClient。
Docker 沙箱层	编译和运行 generator、validator、solution；禁止网络；限制 CPU、内存和时间；只挂载临时目录。

testlib 层	validator.cpp 使用 testlib.h; checker.cpp 暂不实现复杂逻辑，只保留接口。
jngen 层	generator.cpp 使用 jngen.h; 按数据结构选择 array、tree、graph、DAG、matrix、queries 等生成方式。
存储层	SQLite 保存项目、配置、资源限制和状态；本地 storage 保存 generated files、workdirs、datasets 和 reports。
日志层	记录 Dify 调用、Docker 编译、运行、validator 失败和标程运行失败原因。

5.2 Mermaid 架构图描述

```
flowchart TB
    User[教师 / 出题人] --> UI[React + Vite 前端]
    UI --> API[FastAPI 后端 API]
    API --> DB[(SQLite)]
    API --> FS[本地 storage]
    API --> Dify[DifyWorkflowClient / MockDify]
    API --> Sandbox[Docker 沙箱执行器]
    Dify --> Agents[分析 / 识别 / 规划 / 代码草稿 / 修复 / 报告]
    Sandbox --> Gen[generator.cpp + jngen.h]
    Sandbox --> Val[validator.cpp + testlib.h]
    Sandbox --> Sol[solution.cpp 标程]
    Gen --> Inputs[*in]
    Inputs --> Val
    Inputs --> Sol
    Sol --> Outputs[*out]
    Val --> Report[质量报告]
    Outputs --> Package[zip 导出包]
    Report --> Package
```

6 智能体 workflow 设计

6.1 workflow 原则

智能体 workflow 负责分析与辅助生成，不负责最终事实判定。所有 AI 输出均应进入人工可见的确认环节，并通过 Docker 编译、testlib 校验和标程运行结果进行确定性验证。后端通过 Python 封装 DifyWorkflowClient，优先检查本地 dify 文件夹中的已有调用方

式；若不可用，则提供 MockDifyWorkflowClient 保证 MVP 可演示。

环境变量包括:DIFY_BASE_URL、DIFY_API_KEY、DIFY_WORKFLOW_ID、USE MOCK_DIFY。
API Key 不允许写死在代码中。

6.2 节点设计

节点	输入	输出
Solution Input Analysis Agent	C++ 标程代码、数据范围、数量规模、难度等级	输入结构草案、变量范围草案、结构约束草案、隐含条件、不确定字段
Data Structure Recognition Agent	输入结构草案、标程读取逻辑	需要生成的数据结构，如 array、tree、graph、queries
Case Planning Agent	数据范围、数量规模、难度等级、数据结构识别结果	测试点计划，包含 category、scale、purpose、seed、resource limit
Generator Code Agent	测试点计划、数据结构规格	基于 jngen 的 generator.cpp 草稿
Validator Code Agent	输入结构、范围、规模和结构约束	基于 testlib 的 validator.cpp 草稿
Repair Agent	编译错误、运行错误、validator 错误	修复后的代码草稿，受最大重试次数限制
Report Agent	全流程日志、计划、结果、失败原因	质量报告和需求方确认报告草稿

7 Docker 沙箱设计

7.1 镜像内容

Docker 镜像应包含 Linux 基础环境、g++、make、Python 运行时、testlib.h、jngen.h 以及必要的脚本工具。镜像不包含任何业务密钥，不在容器内访问外部网络。

7.2 编译流程

1. 后端为每次任务创建临时 workdir。
2. 写入 generator.cpp、validator.cpp、solution.cpp 和资源头文件。
3. 使用 Docker 运行编译命令，分别生成 generator、validator、solution 可执行文件。
4. 捕获 stdout、stderr、exit code 和编译耗时。
5. 任一核心代码编译失败时停止后续数据生成。

7.3 运行流程

1. 按测试点计划调用 generator，传入 seed、case_id、scale 等参数。
2. 对生成的 .in 执行文件大小检查和基础格式检查。
3. 使用 validator 校验输入文件。
4. 使用 solution 读取 .in 并生成 .out。
5. 记录运行时间、内存、退出码、stdout size、stderr、是否超时、是否内存超限。

7.4 安全限制

- 网络禁用：Docker 运行使用 `--network none`。
- 内存限制：通过 `--memory` 设置容器上限。
- CPU 限制：通过 `--cpus` 设置可用 CPU。
- 文件挂载：只挂载临时工作目录，不挂载项目根目录和用户目录。
- 时间限制：后端对子进程设置 timeout，超时后终止容器。

7.5 错误分类

错误码	含义
COMPILE_ERROR	generator、validator 或 solution 编译失败。
RUNTIME_ERROR	程序运行时异常退出。
TIME_LIMIT_EXCEEDED	运行超过配置时间限制。

MEMORY_LIMIT_EXCEEDED	运行超过配置内存限制。
VALIDATION_FAILED	输入文件未通过 testlib validator。
INPUT_TOO_LARGE	输入文件超过单文件大小限制。
OUTPUT_TOO_LARGE	输出文件超过单文件大小限制。
DOCKER_ERROR	Docker 启动、挂载或资源控制异常。

8 内存控制设计

8.1 资源限制字段

字段	说明
generator_memory_mb	generator 运行内存上限。
validator_memory_mb	validator 运行内存上限。
solution_memory_mb	标程运行内存上限。
generator_timeout_ms	generator 单次运行超时。
validator_timeout_ms	validator 单次运行超时。
solution_timeout_ms	标程单个测试点运行超时。
max_input_file_size_mb	单个输入文件最大大小。
max_output_file_size_mb	单个输出文件最大大小。
max_total_dataset_size_mb	最终数据包总大小上限。
max_retry_count	AI 修复和重新生成的最大重试次数。

8.2 失败停止策略

后端任务采用 fail-fast 策略。标程编译失败、validator 编译失败、generator 编译失败时终止流程。单个测试点生成失败时可按最大重试次数重试；重试后仍失败则标记为 invalid，不进入最终数据包。若合法测试点数量低于用户要求，则报告中明确提示“数据包不满足数量规模要求”。

9 jngen 数据结构生成设计

本项目不按“数组题、图题、字符串题”等题型分类，而按“需要生成的数据结构”分类。jngen README 显示其支持随机数、命令行参数、数组包装、打印器、图和树生成、字符串、几何对象等能力，适合作为复杂结构 generator 的基础库。

结构	适用输入	生成策略	validator 校验点
array	一维数组、权值列表	随机、单调、重复值、极值混合	长度、元素范围、换行和空格
sequence	有序序列、操作序列	递增、递减、震荡、局部有序	顺序关系、长度、范围
permutation	排列、排名、映射	随机排列、逆序、近似有序	元素唯一性、取值为 1..n
string	字符串、模式串	随机字符、重复块、边界长度	字符集、长度、格式
tree	树结构	链、星、随机树、重心偏移树	n-1 条边、连通、无环
graph	一般图	稀疏、稠密、连通、多连通块	n-1 条边、连通、无环
DAG	有向无环图	拓扑序生成、分层 DAG	无环、边方向、点边范围
bipartite graph	二分图、匹配图	两侧规模可控、随机边、极端不均衡	左右集合边界、边合法性
matrix	矩阵、二维权值	随机、行列模式、极值块	行列数、元素范围
grid	网格地图	障碍密度、连通区域、边界墙	行列数、字符集、连通性要求
queries	查询序列	更新/询问混合、极端重复、在线顺序	查询数、操作类型、参数范围
geometry points	平面点集	随机点、共线、凸包、极值坐标	坐标范围、重复点策略
combined structure	图 + 查询、树 + 权值等	分阶段生成并保持关联关系	各子结构约束与跨字段关系

10 testlib validator 设计

testlib README 显示，validator 通过 `registerValidation(argc, argv)` 读取标准输入并严格检查格式、范围、行尾和 EOF；非法输入会返回非零状态并输出错误信息。MVP 中 validator 是主要验题标准，系统基础校验器只做文件大小、空文件、基本可读性等辅助检查。

10.1 validator 职责

- 1. 检查输入格式是否与确认后的输入结构一致。
- 2. 检查整数、字符串、图边、树边、查询参数等是否在数据范围内。
- 3. 检查结构约束，如树连通无环、排列唯一、图边合法、DAG 无环。
- 4. 检查输入结尾，避免多余 token。
- 5. 输出明确错误提示，便于 RepairAgent 或人工修改 generator。

10.2 与基础校验器的关系

系统基础校验器负责轻量检查：文件是否存在、是否为空、是否超过大小限制、是否重复、是否包含不可接受的二进制内容。testlib validator 负责领域语义校验，是判断 `.in` 是否可进入最终数据包的核心依据。

11 数据模型设计

模型	关键字段与说明
Project	project_id、name、description、status、created_at、updated_at。
ProblemInputConfig	standard_code_summary、range_spec、scale_spec、difficulty、confirmed_fields、uncertain_fields。
ResourceLimit	generator/validator/solution 的 memory、timeout、CPU、文件大小和总包大小限制。
StandardSolution	source_path、language、compile_status、compile_log、binary_path。

DataSetStructureSpec	structure_type、variables、constraints、confidence、needs_confirmation。
CasePlan	case_id、category、scale、purpose、seed、resource_limit。
GeneratedCode	code_type、source_path、compile_status、repair_count、latest_log。
TestCase	case_id、input_path、output_path、category、status、hash。
ValidationResult	case_id、basic_check、validator_exit_code、message、passed。
ExecutionResult	case_id、exit_code、runtime_ms、memory_kb、stdout_size、stderr、verdict。
ResourceUsage	case_id、phase、time_ms、memory_kb、file_size_bytes。
WorkflowRunLog	run_id、node_name、input_summary、output_summary、status、error。
ExportPackage	package_id、zip_path、metadata_path、report_path、created_at。

12 API 设计

接口	说明
POST/api/projects	创建项目。
GET/api/projects	获取项目列表。
GET/api/projects/{project_id}	获取项目详情。
PUT/api/projects/{project_id}	更新项目基础信息。
POST/api/projects/{project_id}/solution	上传或保存 C++ 标程。
POST/api/projects/{project_id}/solution/compile	Docker 编译标程。
PUT/api/projects/{project_id}/input-config	保存数据范围、数量规模、难度等级。

GET/api/projects/{project_id}/input-config	查询输入配置。
PUT/api/projects/{project_id}/resource-limits	保存资源限制。
GET/api/projects/{project_id}/resource-limits	查询资源限制。
POST/api/projects/{project_id}/workflow/analyze	运行 Dify / Mock Dify 分析流程。
GET/api/projects/{project_id}/workflow/result	获取 workflow 结果。
POST/api/projects/{project_id}/generate	生成并校验数据。
GET/api/projects/{project_id}/cases	获取测试点列表。
GET/api/projects/{project_id}/cases/{case_id}	获取单个测试点详情。
GET/api/projects/{project_id}/report	获取质量报告。
POST/api/projects/{project_id}/export	导出 zip 数据包。

13 界面设计

页面	页面目标	核心组件与交互
首页 / 项目列表页	管理单题项目	项目表格、状态标签、新建按钮、搜索筛选、最近运行状态。
输入配置页	汇总本题输入入口	标程状态、数据范围文本框、数量规模文本框、难度等级单选下拉框、确认状态。
标程导入页	提供唯一答案来源	代码编辑器、上传按钮、编译按钮、编译日志、错误定位。
数据范围配置页	录入变量范围和结构关系	多行文本框、Constraints 示例提示、变量名一致性提示。
数量规模配置页	配置测试点数量等数值规模	单行文本框、数字格式校验、默认值提示。
难度等级配置页	控制测试强度	单选下拉框、难度说明、覆盖强度预览。

资源限制配置页	控制时间、内存、文件大小	timeout、memory、CPU、输入输出大小、总包大小。
智能体流程页	展示 Dify 节点执行状态	节点时间线、输入输出摘要、不确定字段、重试记录。
测试点计划页	人工确认测试点覆盖	category、scale、purpose、seed 表格，启用/禁用开关。
代码预览页	查看 generator / validator	双栏代码预览、编译状态、修复建议、人工确认按钮。
数据预览页	查看输入输出和日志	测试点列表、输入预览、输出预览、资源使用。
质量报告页	展示最终质量结论	覆盖摘要、失败原因、人工确认事项、导出入口。
导出页	生成本地数据包	zip 内容清单、导出按钮、下载路径、metadata 预览。

13.1 前端输入控件设计

输入配置相关控件应尽量降低用户填写成本，同时保证后端能够稳定解析。数据范围不采用复杂表单拆分，而采用多行文本框，让用户输入接近竞赛题面 Constraints 的文本描述。页面需要给出示例，例如 `2 <= N <= 5 * 10^5、N is an integer、S is a string of length N consisting of o and x`。输入框旁必须提示：变量名需要与标程代码中的变量名保持一致，否则系统分析标程读取逻辑时可能无法对应字段。

数量规模采用单行文本框，用户输入测试点数量等数值规模，前端需要做非空、正整数和合理上限校验。难度等级采用单选下拉框，建议提供“简单”“中等”“较难”三个选项，用户每次只能选择一个难度等级。

配置项	控件形式	交互与校验要求
数据范围	多行文本框	支持类似 Constraints 的自然语言或约束列表；提示变量名必须与标程代码一致；用于 AI 分析输入结构与 validator 草稿。
数量规模	单行文本框	用户输入测试点数量等数值；前端校验正整数、非空值和上限；后端据此生成测试点计划。

难度等级	单选下拉框	从预设难度中选择一个；影响边界数据、极限数据和特殊构造比例。
------	-------	--------------------------------

14 应用功能边界

14.1 建议纳入应用的功能

功能	说明
单题项目管理	创建、查看、编辑单题项目，保存项目状态和基础配置。
标程导入与编译	上传 C++ 标程并在 Docker 中检查是否能编译。
数据范围配置	通过多行文本框录入类似 Constraints 的数据范围说明，变量名需与标程代码保持一致。
数量规模配置	通过单行文本框录入测试点数量等数值规模，并做数字格式校验。
难度等级配置	通过单选下拉框选择难度等级。
AI 分析与规划	分析标程输入结构，生成输入结构草案、数据结构识别结果和测试点计划。
数据结构识别	按 array、tree、graph、queries 等数据结构识别，而不是按题型识别。
输入生成	使用 generator 批量生成 .in。
输入校验	使用 validator 检查格式、范围和逻辑合理性。
标准输出生成	使用用户导入的标程生成 .out。
质量报告	展示覆盖情况、失败原因和资源使用。
zip 导出	导出 .in/.out、代码和报告。

14.2 暂不纳入应用需要二次确认的功能

功能或事项	暂不纳入原因或需确认问题
-------	--------------

OJ 平台兼容	无法理解需求方的“兼容 OJ 平台、课堂练习、模拟比赛”是什么具体需求，需要确认兼容对象是文件命名、zip 结构、metadata、checker / validator，还是具体 OJ 的 API 对接。
课堂练习 / 模拟比赛	暂不纳入当前应用。需确认需求方是否需要教师下载数据、学生在线练习、多题管理、分值、排名和提交记录等完整教学或比赛流程。
SPJ 自动生成	暂不纳入当前应用。特殊评测通常需要完整题面和输出判定规则，仅通过标程代码难以可靠推断。
交互题	暂不纳入当前应用。交互题的执行、输入输出协议和校验机制与普通题不同，需要单独设计。
完整题面辅助输入	当前应用默认不输入完整题面。需确认后续是否允许把题面作为可选辅助信息，以提升 AI 对输入结构和特殊规则的理解。
文件与测试点上限	需确认单题最大测试点数量、单个输入输出文件大小和总数据包大小上限。

15 验收标准

序号	可检查标准
1	能创建单题项目。
2	能导入并编译 C++ 标程。
3	能录入数据范围、数量规模、难度等级。
4	能分析标程输入结构并输出需要人工确认的不确定字段。
5	能生成测试点计划。
6	能生成 generator.cpp。
7	能生成 validator.cpp。
8	generator 和 validator 都在 Docker 中运行。
9	每个 .in 通过 validator。
10	标程能生成 .out。
11	能显示时间、内存、文件大小。

- 12 能导出 zip。
 - 13 能生成 report.md。
 - 14 遇到编译错误、运行错误、超时、内存超限时能显示明确错误。
-

16 推荐技术栈

- 后端：FastAPI。
- 前端：React + Vite。
- 数据库：SQLite。
- 执行环境：Docker。
- 智能体调用：Dify Python client wrapper。
- 输入校验：testlib.h。
- 数据生成：jngen.h。
- 文件存储：本地 storage。
- 导出：Python zipfile。
- Docker 调用：后端 subprocess 调 Docker，但不直接运行用户代码。

17 项目目录结构建议

```
backend/  
  app/  
    api/  
    models/  
    services/  
    sandbox/  
    workflows/  
frontend/  
  src/  
docker/  
third_party/testlib/  
third_party/jngen/  
dify/  
storage/  
projects/
```

```
workdirs/  
datasets/  
reports/  
docs/  
scripts/
```

18 小组分工

方向	工作内容
前端 UI	页面设计、交互流程、代码预览、数据预览、报告展示。
后端 API	FastAPI 接口、数据模型、任务状态、导出逻辑。
Docker 沙箱	镜像、编译运行、资源限制、错误分类。
Dify 工作流	Client 封装、Mock 实现、节点输入输出、日志记录。
jngen generator	数据结构生成策略、seed/case_id、生成参数。
testlib validator	输入格式、范围、结构约束、错误提示。
报告与答辩	设计报告、需求确认报告、演示脚本、PPT 素材。

19 开发里程碑

时间	任务
Day 1	需求确认、架构、报告初稿。
Day 2	前后端骨架、数据库模型。
Day 3	Docker 沙箱、标程编译运行。
Day 4	jngen generator、testlib validator。
Day 5	Dify / Mock Dify 工作流。
Day 6	数据生成、校验、报告、导出。
Day 7	联调、界面优化、演示准备。

20 参考资料与当前目录依据

- `goal.md`: 定义项目定位、MVP 边界、流程、架构、API、界面和验收要求。
- 课程考查要求.pdf: 明确项目设计报告需要覆盖业务逻辑、功能设计、交互界面和二次需求确认。
- 项目要求.jpg: 明确项目题目和主要功能, 包括标程代码、数据范围、数量规模、难度等级、数据生成、数据校验和导出。
- `testlib/README.md`: 说明 `testlib` 在 Codeforces 等竞赛中的使用, 并提供 `validator`、`checker`、`generator` 的典型模式。
- `jngen/README.md`: 说明 `jngen` 支持随机数、参数解析、数组、图、树、字符串和几何等生成能力。
- `dify/README.md`: 说明 Dify 是开放的 LLM 应用开发平台, 支持 Workflow、Agent、API 和 Docker Compose 本地部署。
- `OI-wiki/docs/tools/polygon.md`: 说明 Polygon 中 `validator`、`generator`、`checker`、`solutions`、`Tests`、`Invocations`、`Packages` 等出题验题流程。